

AI Agents



Content

- What is AI Agent?
- Model
- Orchestration
- Tools
- Multi-Agent System
- Managing risks in AI Agents

INTTRVU.ai

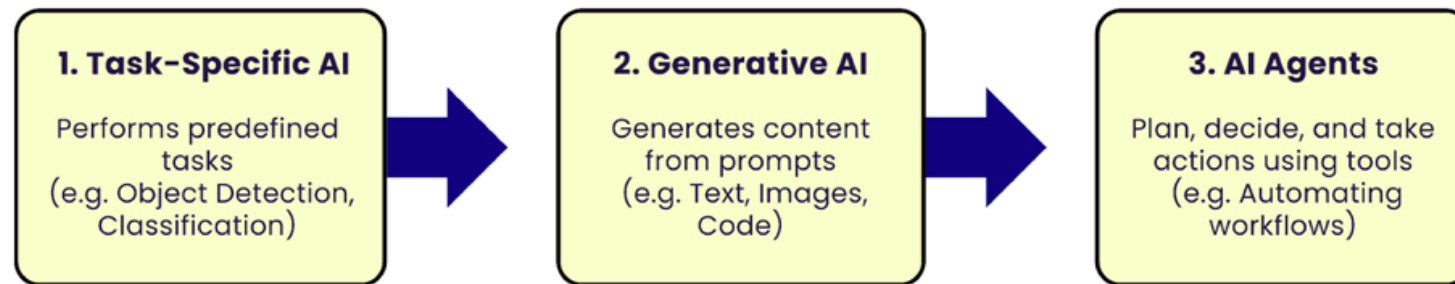
Website: <https://inttrvu.ai>

What is AI Agent?

Website: <https://inttrvu.ai>

What is AI, Gen AI and AI Agents?

Concept Flow : AI Autonomy Progression



Increasing Autonomy and Control

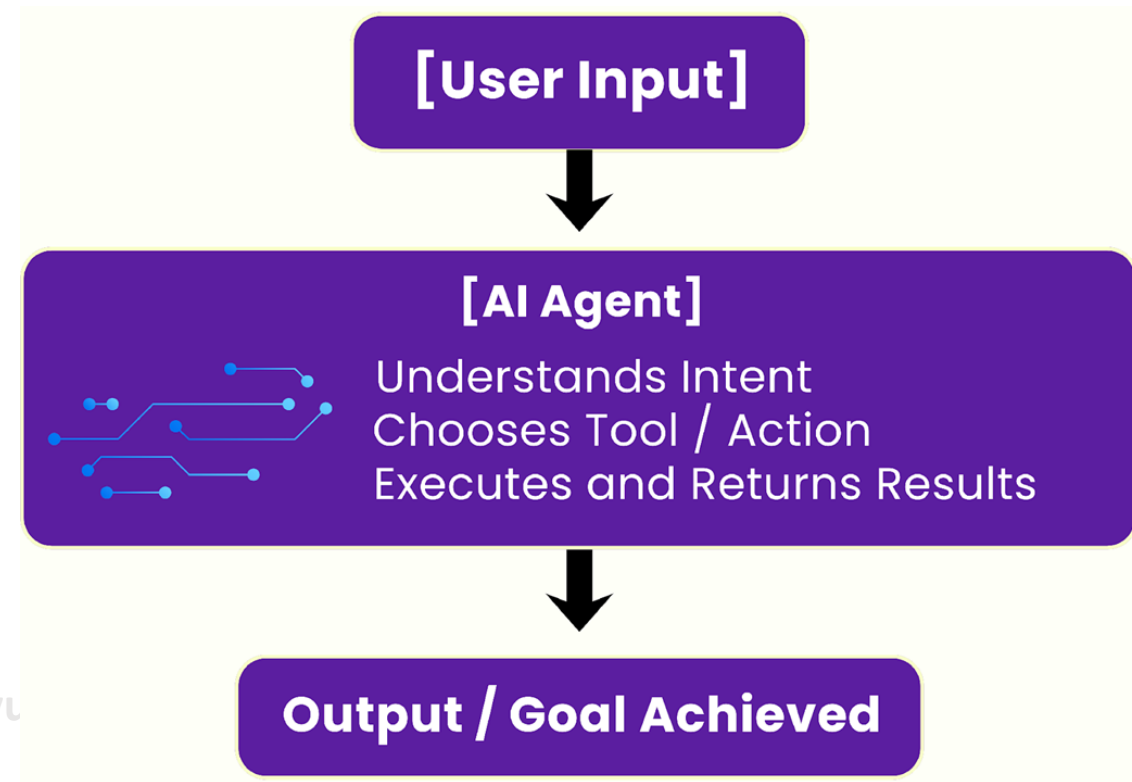
Type	Description	Example
AI	Performs tasks requiring human intelligence	Object detection from CCTV footage
GenAI	Generates new content (text, image, video)	ChatGPT (text), DALL·E (image)
AI Agent	Autonomous system that reasons, plans, and acts with minimal input	AI planner for vehicle launch or task automation

What is AI Agent?

An **AI Agent** is a system or application that **perceives its environment**, **processes information**, and **takes actions autonomously** to achieve specific goals using the **tools, data, or APIs** available to it.

Examples:

1. **Customer Support Agent:** Handles bookings, cancellations, or reschedules using internal databases.
2. **Database Query Agent:** Converts natural-language questions into SQL queries to retrieve accurate data.
3. **Travel Booking Agent:** Books or modifies flight reservations based on a conversational command.



What is AI Agent?

AI agents consist of three main components

Model:

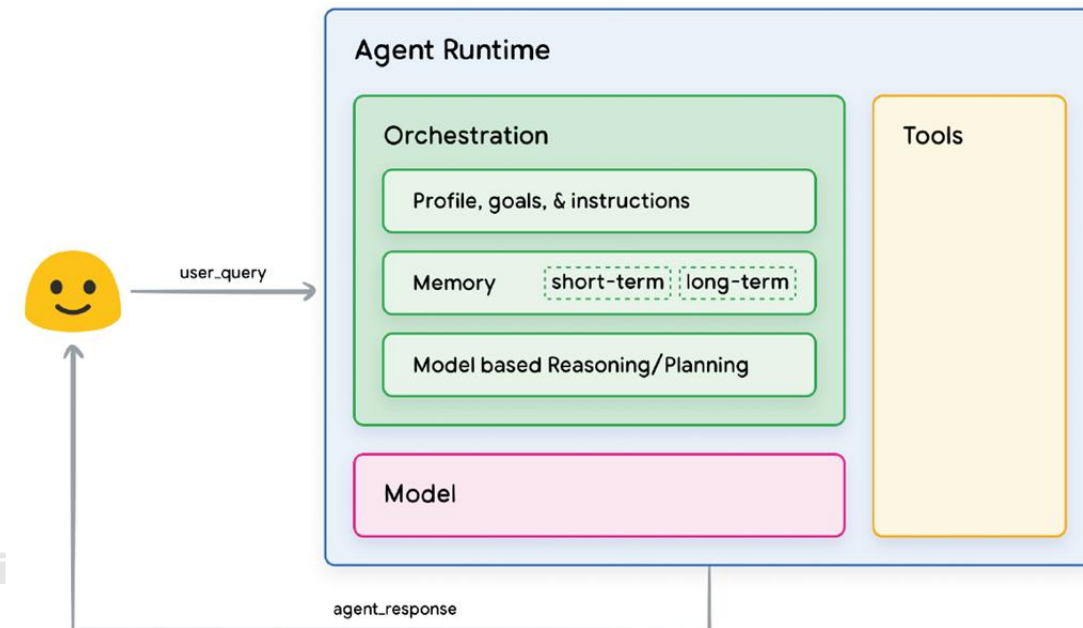
- ☐ E.g., Language model (e.g., GPT4, Llama 3) capable of understanding and responding in natural language

Orchestration:

- ☐ It defines the sequence of actions the AI agent should take to solve a problem

Tools:

- ☐ A set of functions/APIs for automating the process
- ☐ Empowers AI agents to interact with external data and services, e.g., Google API for flight booking



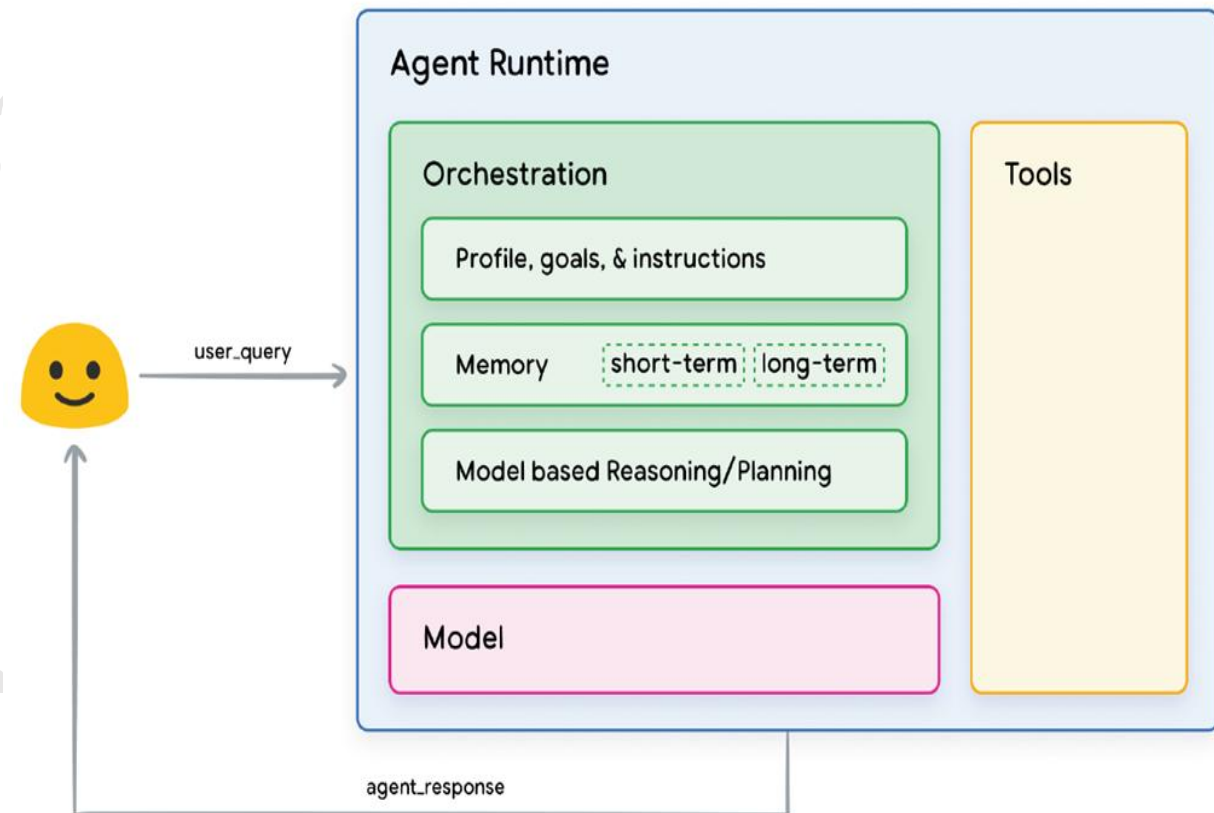
Model in AI Agents

Website: <https://inttrvu.ai>

Model: AI capabilities

Key Capabilities of Language Models

- **Natural Language Understanding (NLU):** Comprehends user queries, context, and intent.
- **Reasoning & Decision-Making:** Infers logical steps or conclusions from input data.
- **Tool Use & API Integration:** Can invoke functions, APIs, or ML models to perform tasks.
- **Knowledge & Context Recall:** Retains session context to make consistent, context-aware responses.



Models vs Agents

Models	Agents
Knowledge is limited to what is available in their training data.	Knowledge is extended through the connection with external systems via tools
Single inference/prediction based on the user query. No management of session history or continuous context unless explicitly implemented	Performs actionable tasks based on the orchestration layer
No native tool implementation.	Tools are natively implemented in agent architecture.
E.g., GPT3, GPT4	E.g., Agent to block a transaction with financial fraud, Customer support chatbot which can do flight bookings, cancellations etc.

Orchestration

Website: <https://inttrvu.ai>

Orchestration: Defining Action Sequences

Orchestration refers to how an **AI Agent determines and executes the sequence of actions** required to achieve a goal — deciding *what to do next*, *which tool to use*, and *when to act*.

Type	Who Defines the Workflow?	Example	Classification
Manual Orchestration	Programmer pre-defines the sequence of actions	A pipeline where the developer codes each step explicitly	Compound AI System
Dynamic Orchestration	The LLM decides the workflow at runtime based on the goal and context	An agent that autonomously plans and executes tasks	True Agentic System

Orchestration: Defining Action Sequences

If the flow is fully or mostly manual, the system is not truly an **AI agent**—it's more like a compound AI System (AI-powered workflow), but not an AI agent

Characteristic	AI-Powered Workflow	AI Agent (Agentic System)
Flow defined manually	✓ Yes	✗ No – LLM determines steps dynamically
LLM used for isolated tasks	✓ Summarization, classification, etc.	✓ And also for planning + decision-making
Goal-directed behavior	✗ Fixed pipeline	✓ Has goals, adapts actions accordingly
Reactivity & autonomy	✗ Limited	✓ High – reacts to environment/state

Orchestration: Defining Action Sequences

What is workflow?

- A workflow is like a recipe: “Do Step 1 → Step 2 → Step 3.”
- An agent is like a chef who asks: “What dish should I make with what's in the fridge?” Then plans, adapts, and executes based on the goal.

When Is It an Agent?

It's an agent if the LLM is empowered to:

1. Understand goals
2. Choose what steps/tools to use
3. Execute, reflect, and adapt in a loop

website: <https://inttrvu.com>

Workflow System	Agentic System
Follows fixed steps	Decides steps dynamically
Predefined logic	Adaptive decision-making
No self-correction	Reflects and adjusts
Developer controls flow	LLM controls flow
Example: RAG chatbot	Example: AI task planner

How AI Agents Define Their Own Flow

Goal Driven Planning

- Receives a high level goal
- Breaks it into logical steps dynamically

Example:

Book flight → Check dates → Search → Compare → Confirm → Book

Tool Selection

- Has access to APIs, search, databases, calculators
- Chooses the right tool at each step

Example:

Use flight_search → Sort by price → Return best option

How AI Agents Define Their Own Flow

Dynamic Execution

- Executes plan step by step using tools
- Adjusts or replans if results change or fail

Example:

No results → Try nearby airports → Use flexible dates

Reflection & Memory

- Reviews its own progress and improves decisions
- Uses memory to personalize across sessions

Example:

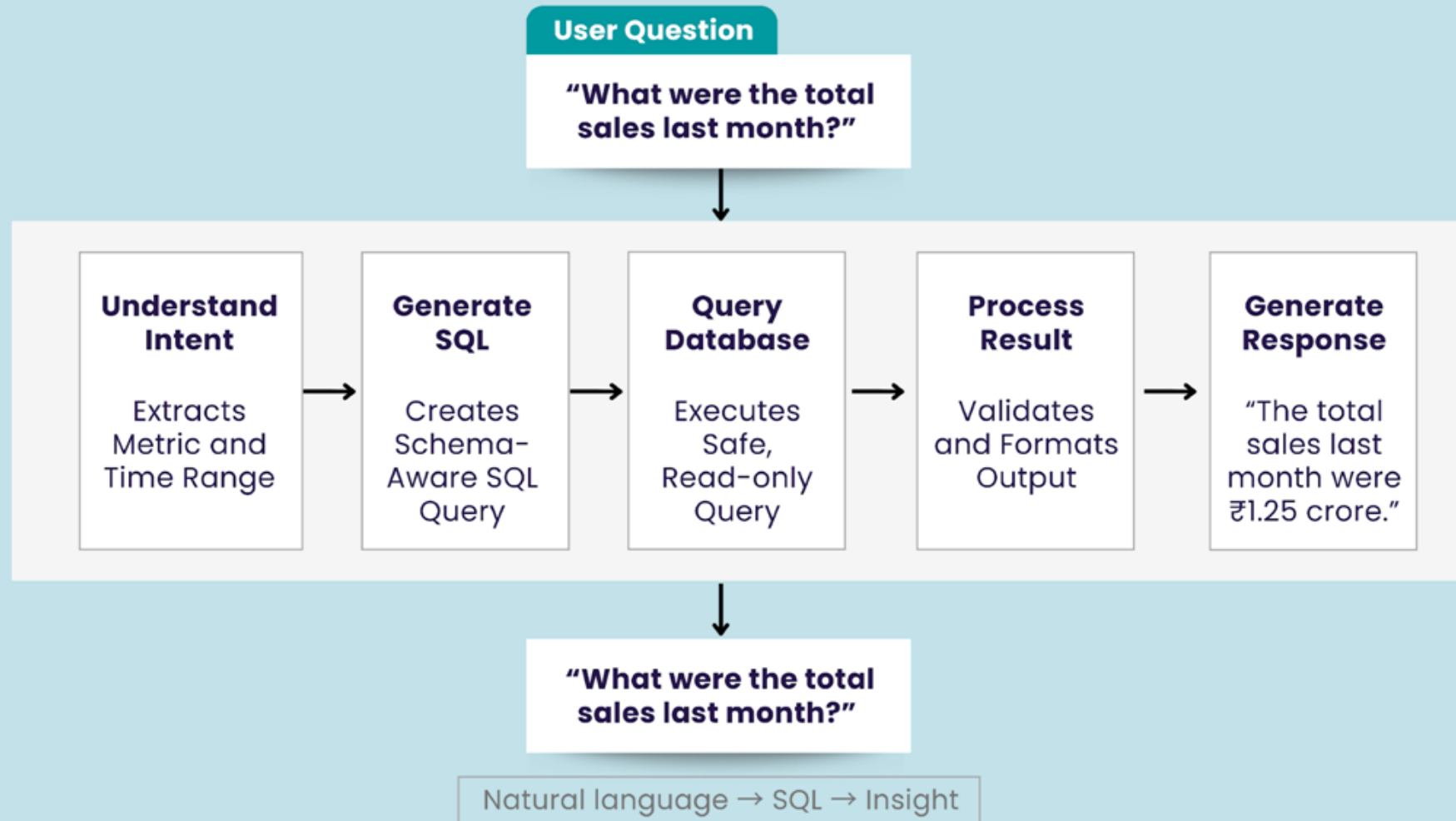
User prefers Vistara → Prioritize Vistara

Website: <https://inttrvu.ai>

Orchestration: Defining Action Sequences

Orchestration defines the **sequence of actions** the AI agent performs to solve a problem — deciding *which step happens next* and *which tool to invoke* at each stage.

Orchestration in Agent for interacting with the SQL database based on natural language questions asked by users:



Reasoning Paradigms for Orchestration

Two main strategies guide how an LLM-based agent defines orchestration

- ☐ ReAct (Reasoning and Action)
- ☐ ReWOO (Reasoning WithOut Observation)

Website: <https://inttrvu.ai>

Reasoning Paradigms in Agents

ReAct (Reasoning and Action)

Combines thought (reasoning) and action (tool use) in a loop.

The agent thinks: “What do I need to do next?”

Takes an action (e.g., calls a tool).

Observes the result.

Thinks again, then acts again.

Thought → Action → Observation → Thought → ...

Website: <https://inttrvu.ai>

Example:

- ❓ Thought: I need flight options for next weekend
- ❓ Action: Call flight_search API
- ❓ Observation: Got 5 options under ₹10K
- ❓ Thought: Let's pick one and book it

Reasoning Paradigms in Agents

ReWOO (Reasoning WithOut Observation)

The agent plans everything up front without seeing intermediate results.

Useful when API calls are expensive or observation is restricted.

Encourages LLM to simulate or plan the whole sequence before acting.

Thought → Thought → Thought → Execute Plan

Example:

- ? Step 1: I'll search flights
- ? Step 2: I'll sort them by price
- ? Step 3: I'll book the cheapest
- ? Then executes full plan in one go

Tools

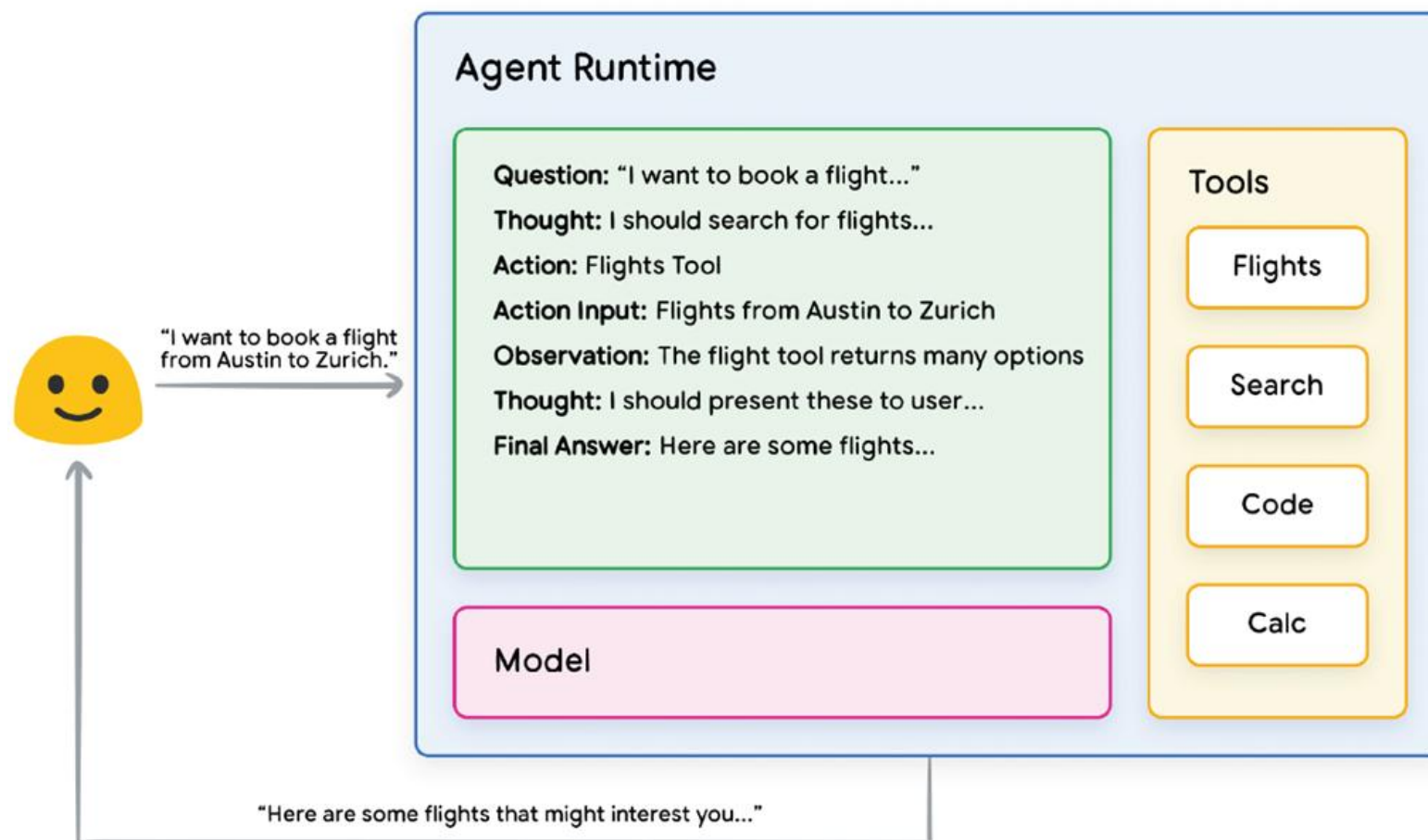
Website: <https://inttrvu.ai>

Tools: Empowering with External Interactions

Tools are external **functions, APIs, or software components** that an AI Agent uses to **interact with data sources, perform actions, or automate workflows** beyond its internal reasoning ability.

Tool Type	Purpose	Examples
Extensions / Plugins	Add capabilities beyond core model (e.g., search, APIs)	Web search, code execution, flight booking APIs etc.
Functions	Reusable functions that agents can call via tool use	<code>get_weather()</code> , <code>query_db()</code> , <code>get_prediction()</code> etc.
Data Stores	Persist and retrieve long-term knowledge or context	Vector DBs (Pinecone, FAISS), SQL, NoSQL

Flight booking Agent using ReAct



Tools: Empowering with External Interactions

Example: Tools in a Flight-Booking AI Agent

Tool Type	Purpose	Example / Technology
Chat Interface (UI Tool)	Enables user interaction via natural language	<i>React.js, Rasa</i>
Natural Language Processing (NLP)	Understands user intent, entities, and context	<i>spaCy, GPT-4, LangChain NLP</i>
Flight Data Integration	Fetches real-time flight details, pricing, and schedules	<i>Google Flights API, Skyscanner API</i>
Backend Tool	Handles API calls, sessions, and service orchestration	<i>Node.js, FastAPI, Flask</i>
Payment Tool	Processes secure flight payments and verifications	<i>Stripe, PayPal</i>

When should we build Agents?

Complex Decisions	Rule Complexity	Unstructured Data
Contextual reasoning	Replace brittle rules	Understand natural language
Refund approvals	Security reviews	Insurance claims

WHEN TO USE AI AGENTS

Complex Decision-Making

Workflows involving nuances judgement, exceptions or context-sensitive decisions
e.g. Refund approval in customer services

DIFFICULT-TO-MAINTAIN RULES

Systems that have become unwieldy due to extensive and intricate rulesets, making updates costly or error-prone
e.g. Performing vendor security reviews

HEAVY RELIANCE ON UNSTRUCTURED DATA

Scenarios that involve interpreting natural language, extracting meaning from documents, or interacting with users conversationally
e.g. Processing a home insurance claim

Multi Agent System

Website: <https://inttrvu.ai>

Multi Agent System

Single-Agent System

- One AI agent handles planning, reasoning, and tool use
 - Executes tasks end-to-end within a single loop
- Example: A personal assistant managing scheduling, reminders, and emails

Multi-Agent System

- Multiple specialized agents collaborate
- Each agent has a defined role and delegates tasks

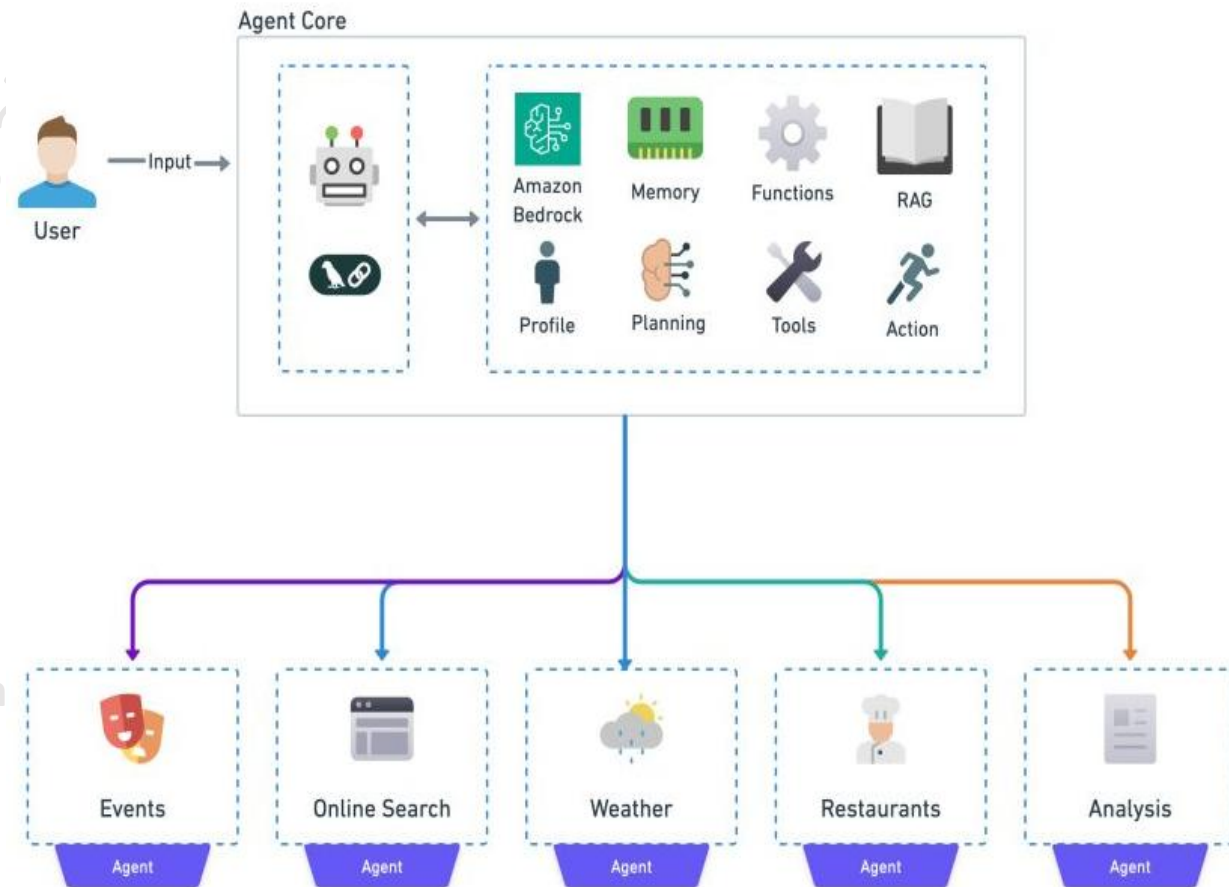
Example:

Planner → Defines tasks

Research → Gathers data

Analysis → Interprets results

Report → Summarizes findings



When should we build multi agent systems?

Complex Logic

- Too many conditional branches make prompts hard to scale
- Split logical segments across separate specialized agents

Tool Overload

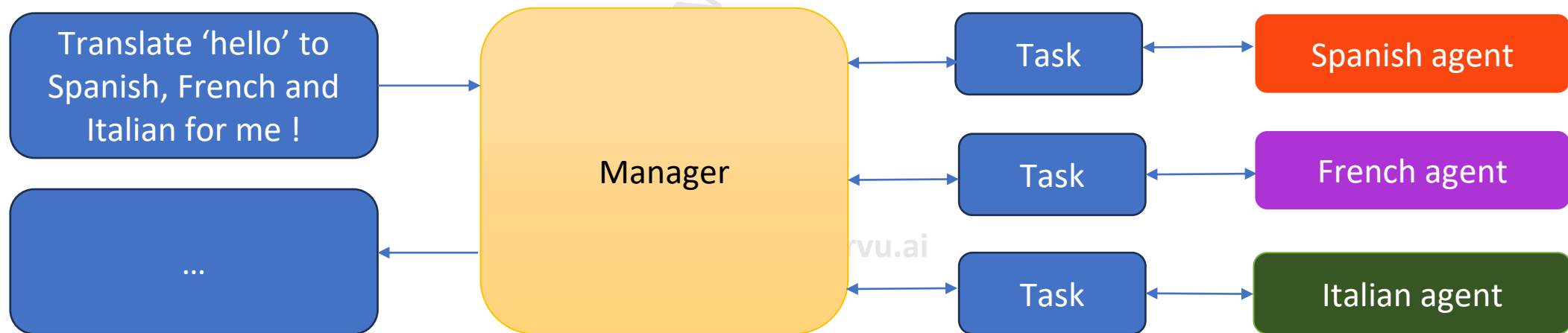
- Too many tools in one agent increases complexity
- If better tool descriptions do not improve performance, divide tasks across dedicated agents

Website: <https://inttrvu.ai>

Manager Pattern

The manager pattern empowers a central LLM—the “manager”—to orchestrate a network of specialized agents seamlessly through tool calls.

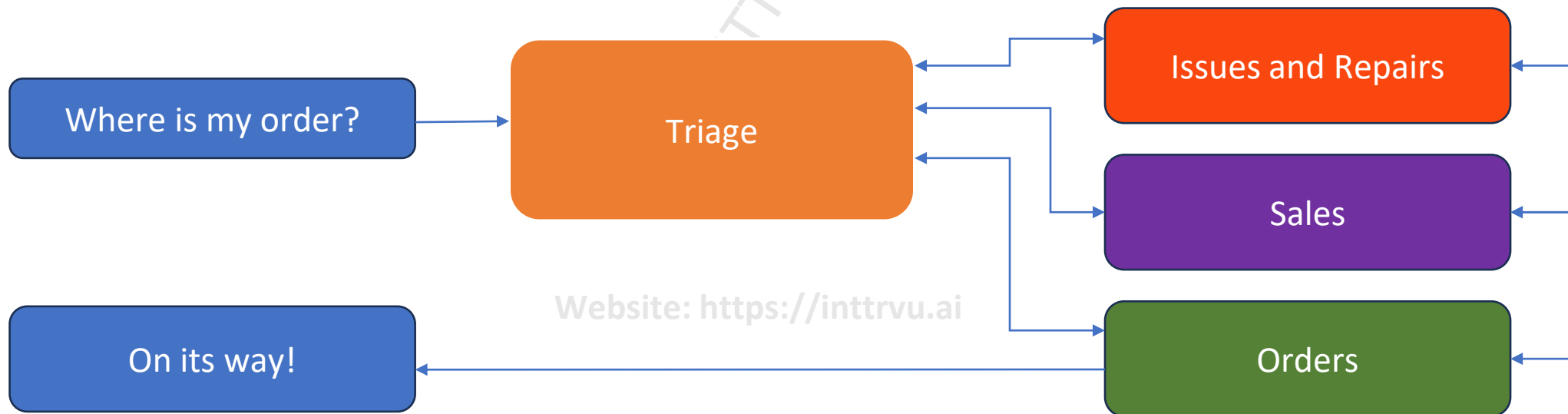
Instead of losing context or control, the manager intelligently delegates tasks to the right agent at the right time, effortlessly synthesizing the results into a cohesive interaction.



Decentralized Pattern

This pattern involves using many agents on equal footing, where one agent can directly hand off workflow control to another agent.

This is optimal when you don't need a single agent maintaining central control or synthesis, instead of allowing each agent to take over execution and interact with the user as needed



Managing Risk in AI Agents

Website: <https://inttrvu.ai>

Guardrails

Guardrails are mechanisms that **manage or reduce risks** associated with AI agent behavior, ensuring **safe, reliable, and compliant execution**.

Purpose of Guardrails

- **Data Privacy:** Prevent leakage of confidential or system prompt information.
- **Brand Alignment:** Enforce consistent tone, values, and ethical behavior.
- **Risk Mitigation:** Avoid unauthorized actions or unintended consequences.

Example: Customer Service Agent

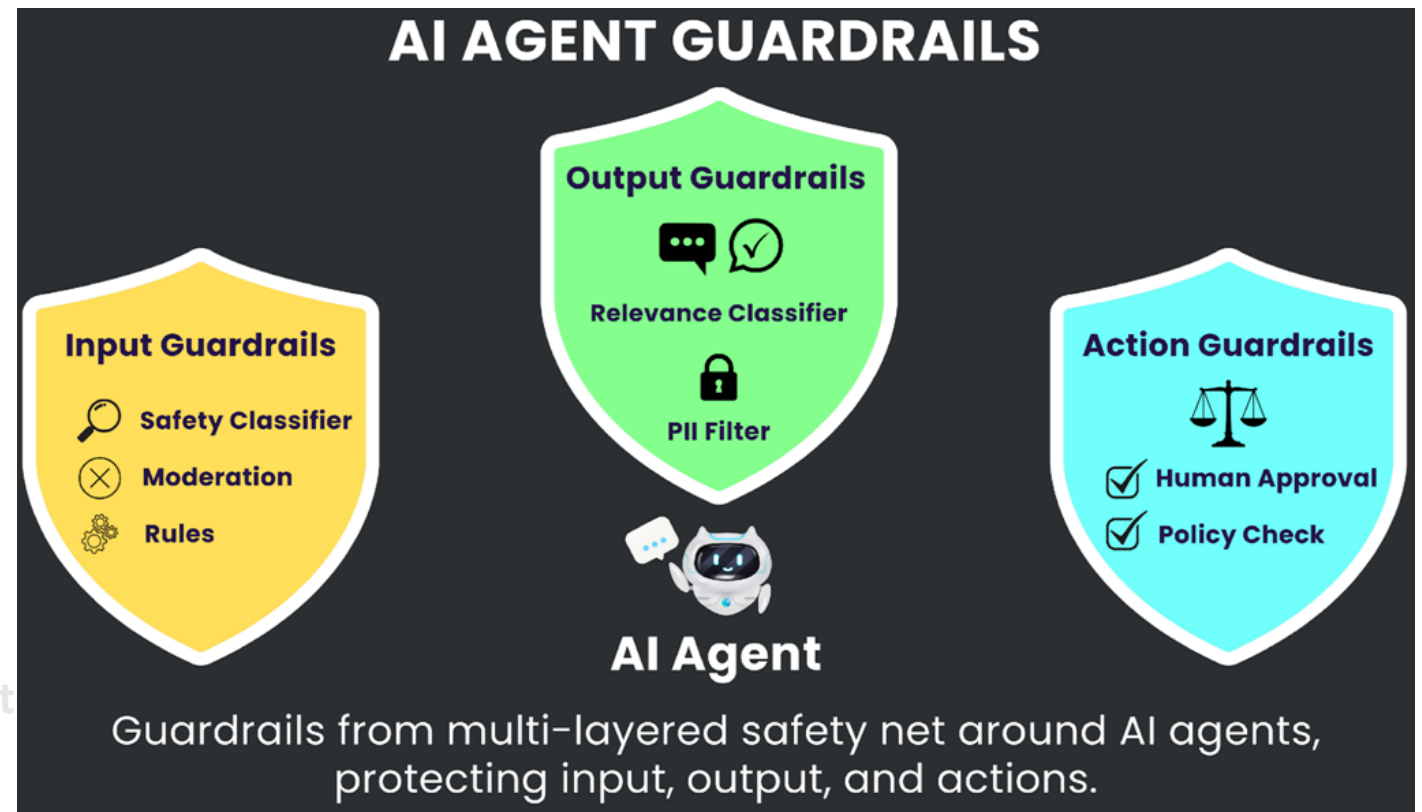
Before processing refunds, the system runs guardrails:

- Verify user identity
- Check refund policy eligibility
- Escalate to human if uncertain

Website: <https://inttrvu.ai>

Types of Guardrails

- 1. **Input guardrails** → keep harmful data *out*
- 1. **Output guardrails** → ensure safe, relevant responses *outgoing*
- 1. **Action guardrails** → control *what the agent actually does*



Types of Guardrails

Guardrail Layer	Purpose	Example Guardrails
Input Guardrails	Protect the system from unsafe, irrelevant, or malicious inputs before processing.	- Safety Classifier: Detects jailbreaks / prompt injections. - Moderation: Flags hate speech, harassment, or violence. - Rules-based Protections: Blocklists, regex filters, input length limits.
Output Guardrails	Review and filter model responses before presenting to users.	- Relevance Classifier: Ensures responses stay on-topic. - PII Filter: Removes personally identifiable information.
Action Guardrails	Add verification before an agent performs real-world operations (API calls, transactions, etc.).	- Policy/Rule Enforcement: Confirm before executing sensitive actions (e.g., processing refunds, database writes). - Human-in-loop Check: Request confirmation on critical decisions.

Human Intervention

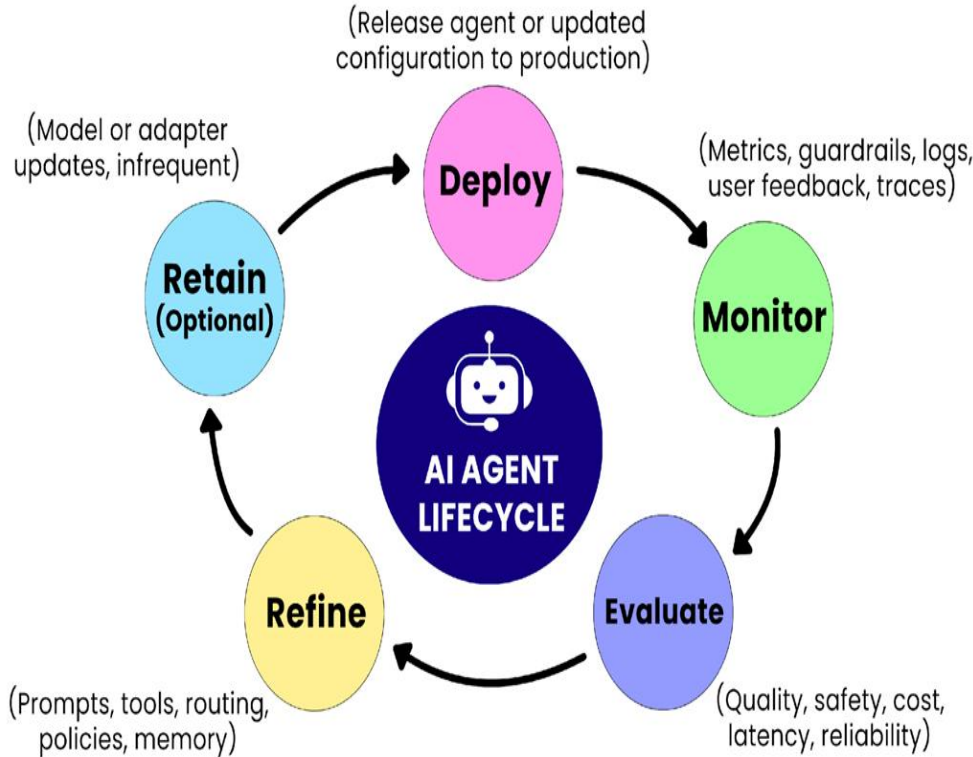
Autonomous agents must know when **to hand control back to a human**.

Human intervention ensures **trust, safety, and accountability** in agent workflows

Trigger Type	Description	Example
Exceeding Failure Thresholds	The agent fails repeatedly or cannot interpret intent after multiple retries.	Customer support bot fails to understand user query → Escalates to human agent.
High-Risk / Sensitive Actions	Any action that could cause financial loss, security breaches, or irreversible changes.	Processing a large refund, canceling an order, or executing payments.
Low Confidence or Ambiguity	When model confidence in decision or classification drops below a set threshold.	The model is unsure about detecting fraud in a transaction → Requests human verification.

Monitoring and Evaluation of AI Agents

Aspect	Description	Example Metrics
Performance Monitoring	Track how efficiently the agent completes tasks and handles requests.	Response time, task completion rate, API success ratio.
Quality Evaluation	Assess the correctness, relevance, and tone of outputs.	Accuracy of SQL queries, relevance of chatbot responses, adherence to brand voice.
Error & Escalation Tracking	Log and review instances where guardrails triggered or humans intervened.	No. of human escalations, frequency of rule violations, failed tool calls.
Continuous Learning & Feedback	Incorporate human feedback and user satisfaction to refine model behavior.	Post-interaction feedback, reinforcement updates, adaptive prompt tuning.



Platforms for Building AI Agents

n8n

Low-code workflow automation with AI agents and tool integrations

Agno

Code-first framework for agent workflows, memory, and multi-agent systems

OpenAI Agents SDK

Full platform for building, orchestrating, and managing agents

Choose based on stack:

- Low-code → n8n
- Code-first → Agno or OpenAI SDK
- Enterprise → Focus on memory, orchestration, and guardrails

Platforms for Building AI Agents

Name	Type / Focus	Key Notes
LangChain	Developer-framework (LLM workflows)	Widely used for building agentic applications with tools, chains, memory. (plivo.com)
LangGraph	Developer-framework (graph/workflow)	Built for stateful multi-actor workflows, DAG style orchestration. (Infosys)
CrewAI	Multi-agent teamwork framework	Role-based agents working in collaboration/teams. (LinkedIn)
AutoGen	Multi-agent orchestration framework	Open-source, built for agent communication and automation. (Anaconda)
Pydantic AI	Developer-framework (type-safe)	Focused on structured outputs/validation + agentic features. (memari-majid.github.io)
LightAgent	Lightweight open-source framework	Recently proposed for production-level simplicity in agentic workflows. (arXiv)
AgentFlow	Low-code multi-agent platform	Designed for multi-agent coordination, enterprise workflows (emerging). (Medium)
AutoAgent	No-code agent-building framework	Enables non-coders to build and deploy agents via natural language. (arXiv)

Real World Use Cases

Domain	Example Use Case	Description
Customer Support	Virtual Helpdesk Agents (e.g., Intercom Fin, Zendesk AI Agent)	Handles FAQs, escalations, and sentiment-based routing with contextual memory.
Healthcare	AI Medical Assistant (e.g., Hippocratic AI, Babylon Health)	Supports doctors by summarizing patient data and suggesting diagnostics.
Finance	Investment Advisor Agent	Analyses market trends, portfolio risks, and executes trades autonomously.
E-Commerce	Product Recommendation Agent	Suggests personalized products based on behavior, reviews, and conversation history.
Education	Intelligent Tutoring Agents	Adaptive learning assistants that personalize study plans and quizzes.
Software Development	Code Assistant Agents (e.g., GitHub Copilot, Codeium)	Automates repetitive coding, debugging, and documentation generation.
Research & Knowledge Work	Autonomous Research Agents (e.g., Agno, AutoGen)	Conduct literature searches, summarize insights, and propose hypotheses.

The Road Ahead — Future of AI Agents

AI Agents are expected to become more autonomous, collaborative, and integrated into our daily lives.



Thank You !

Website: <https://inttrvu.ai>